Messaging and Protocols

Internet Technology

ITECH

消息传递与协议

互联网技术

信息技术

By the end of this week…

By the end of Week 4 you should be able to:

- Explain the fundamentals of network communication
- Understand HTTP and its key characteristics
- Differentiate between HTTP GET and POST
- Understand Inter-Process Communication and specifically RESTful APIs
- Use forms and URLs in Django (own-study, TwD Chapters 7 & 8)

到本周末为止……

到第4周结束时，您应该能够：

- 解释网络
  通信的基本原理
- 理解HTTP及其关键特性
- 区分HTTP GET和POST
- 理解进程间通信和
  特别是RESTful API
- 在Django中使用表单和URL（自学，TwD
  第7章和第8章)

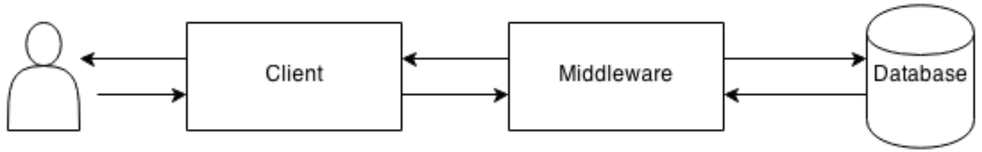
Resources

- This lecture uses some material from the book "Computer Networking: A Top-Down Approach", 8th edition, by Jim Kurose & Keith Ross (Pearson, 2020) and book powerpoint slides: https://gaia.cs.umass.edu/kurose_ross/ppt.php
- If you want to dive deeper into networking and the Application Layer, see Chapters 1 and 2 from the book above

资源

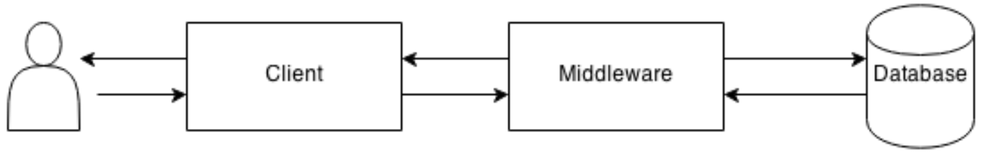
- 本讲座使用了一些来自书籍《计算机网络：自顶向下的方法》第8版的材料，作者为Jim Kurose和Keith Ross（Pearson，2020年）。
  和书籍的PowerPoint幻灯片：
  https://gaia.cs.umass.edu/kurose_ross/ppt.php
- 如果你想更深入地了解网络和应用层，请参见第1章和第2章书籍以上

System Architecture



- We have not yet specified:
  - how messages are going to be sent, and
  - in what format these messages are going to be.
- Different interactions will require using different message formats and protocols.

系统架构



- 我们尚未指定：
  - 消息将如何发送，以及
  - 这些消息将以何种格式呈现。
- 不同的交互将需要使用不同的消息格式和协议。

What happens when you click a URL?

Client

Request

The user types a URL: https://www.gla.ac.uk/study

Client

Response

The webpage is displayed in the browser

当你点击一个

网址时会发生什么?

客户端

请求

用户输入一个网址：https://www.gla.ac.uk/study

客户端

响应

网页在浏览器中显示

A Web Application is a Network Application

Clients:

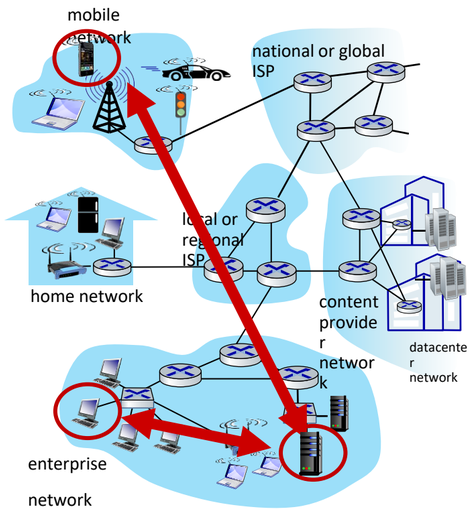
- Contact, communicate with server
- Intermittently connected
- Do NOT communicate directly with each other

Servers / Middleware / Database:

- Always-on host
- Often in datacenters for scaling

Clients **communicate** with the middleware over the **network**

- Message exchange!
- Messages follow rules!



网络应用程序是一个网络应用

客户端:

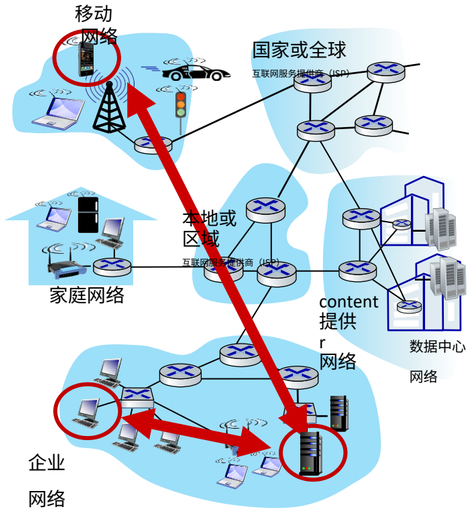
- 联系，与服务器进行通信
- 间歇性连接
- 不直接相互通信

服务器 / 中间件 / 数据库:

- 始终在线的主机
- 通常在数据中心进行扩展

客户端通过网络与中间件进行通信

- 消息交换!
- 消息遵循规则!



The TCP/IP model: 4 layers for network communication

TCP/IP Model

Application
Format data so it can be processed by the application (client)
Protocols:
- **HTTP/HTTPS:** Webpages
- **SMTP, IMAP, POP3:** Emails
- **FTP:** Files
- **DNS:** URL to IP translation

Transport
Split data to packets/Check packets
Protocols:
- **TCP:** Reliable packet transmission
- **UDP:** Unreliable packet transmission

Internet
Assign IP address and route packets
Protocols:
- **IP:** Deliver packets according to IP address

Link/Physical
Prepare packets for transmission over the physical network

We will discuss:
- Application Layer
  - HTTP
  - DNS
- Some elements of the transport and internet layer
  - TCP/IP

The TCP/IP model: 4层网络沟通

TCP/IP模型

应用
将数据格式化，以便应用程序（客户端）可以处理
协议：
- **HTTP/HTTPS: 网页**
- **SMTP, IMAP, POP3: 电子邮件**
- **FTP:** 文件
- **DNS:** URL到IP的转换

传输
将数据分割成数据包/检查数据包
协议：
- **TCP:** 可靠的数据包传输
- **UDP:** 不可靠的数据包传输

互联网
分配IP地址并路由数据包
协议：
- **IP:** 根据IP地址传送数据包

链接/物理
为在物理网络上进行传输准备数据包

我们将讨论：
- 应用层
  - HTTP
  - DNS
- 传输层和互联网层的一些元素
  - TCP/IP

Protocols

Protocols are set of conventions that determine how data is formatted, transmitted, and received over the network

- **Type of messages**, e.g. request/response
- **Message syntax**, e.g. fields in messages
- **Message semantics**, e.g. what fields mean
- **Rules** for when and how processes send & respond to messages

协议

协议是一组约定，决定了
数据的格式、传输方式和
通过网络接收

- 消息类型，例如请求/响应
- 消息语法，例如消息中的字段
- 消息语义，例如字段的含义
- 规则 规定了进程何时以及如何发送和响应消息

Request – Response Pattern

- A **Request-Response** pattern is a way to exchange messages.
  - A requester sends a request message (for a resource or an action)
  - The receiver of that message provides a message in response (with the resource, action, or just an acknowledgement of reception)
- Ideally, this should be performed in a synchronous fashion, however, it is asynchronous

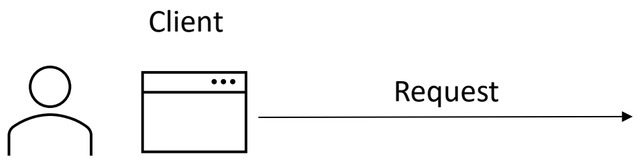
请求– 响应模式

- 请求-响应模式是一种交换消息的方式。
  - 请求者发送请求消息（用于资源或操作）
  - 该消息的接收者提供了一个消息作为回应（包含资源、行动或仅仅是一个收件确认）
- 理想情况下，这应该以同步的方式进行，然而，它是异步

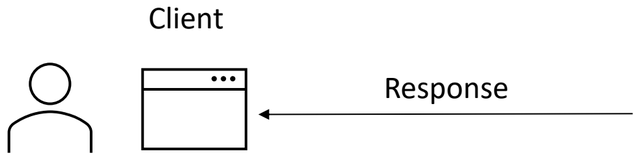
What happens when you click a URL?

当你点击一个

网址时会发生什么?

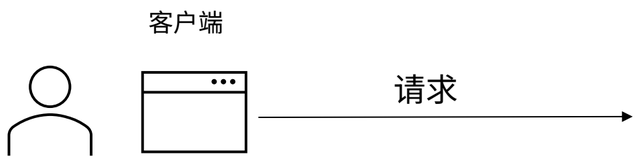


The user types a URL: https://www.gla.ac.uk/study

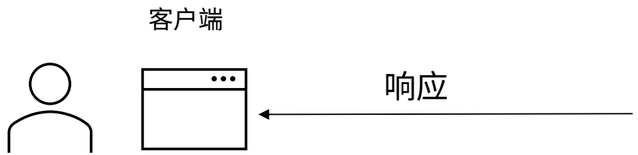


The webpage is displayed in the browser

- What type of request/response is this?
- What messages are exchanged before the webpage is displayed?
- What is the flow of messages?
- What protocols are involved?



用户输入一个网址：https://www.gla.ac.uk/study



网页在浏览器中显示

- 请求/响应是什么类型的这?
- 消息是什么在之前交换网页已显示?
- 什么是 什么是消息?
- 涉及哪些协议?

Components of a URL

URL stands for Uniform Resource **Locator**

- An example: https://www.gla.ac.uk/study

Protocol	Domain	Path to resource
https://	gla.ac.uk	/study

- **HTTPS:** The secure version of the HTTP protocol
- **Domain:** The location of the web application (where the server, middleware, application logic live)
- **Path:** The location of the resource

URL的组成部分

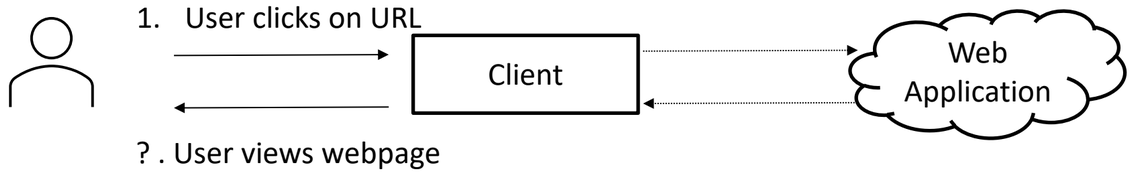
URL代表统一资源定位符

- 示例： https://www.gla.ac.uk/study

协议	域名	路径到资源
https://	gla.ac.uk	/study

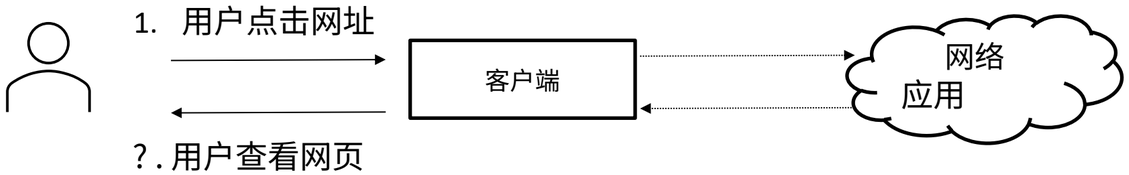
- **HTTPS:** HTTP协议的安全版本
- 域名： 网络应用程序的位置（服务器、中间件、应用逻辑所在的位置）
- 路径： 资源的位置

Clicking on a URL – HTTP messages



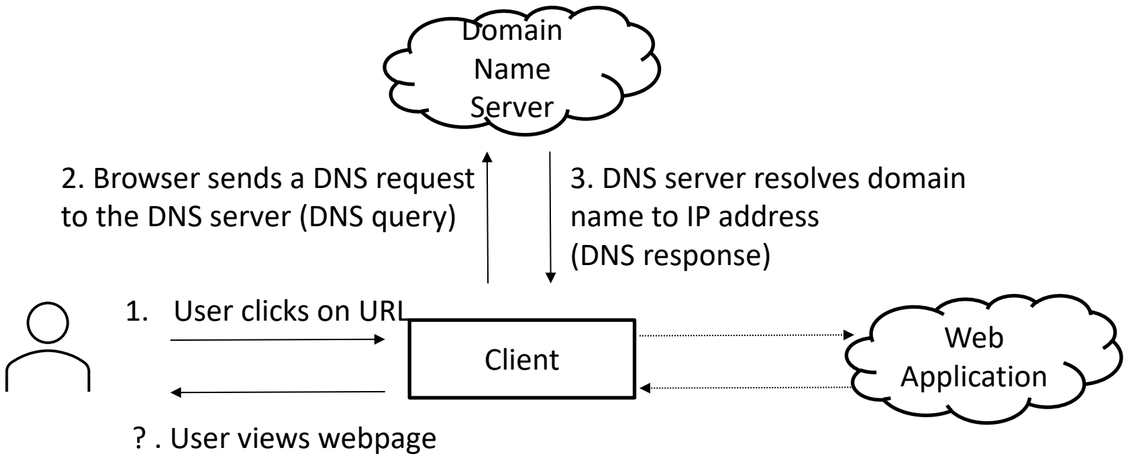
- What is the **location** of the Web Application?
  - The **Domain name** (e.g. gla.ac.uk), but the internet's languages is **IP addresses**
  - **DNS: Domain Name System** – A DNS server will translate the domain name to an IP address

点击网址–HTTP 消息



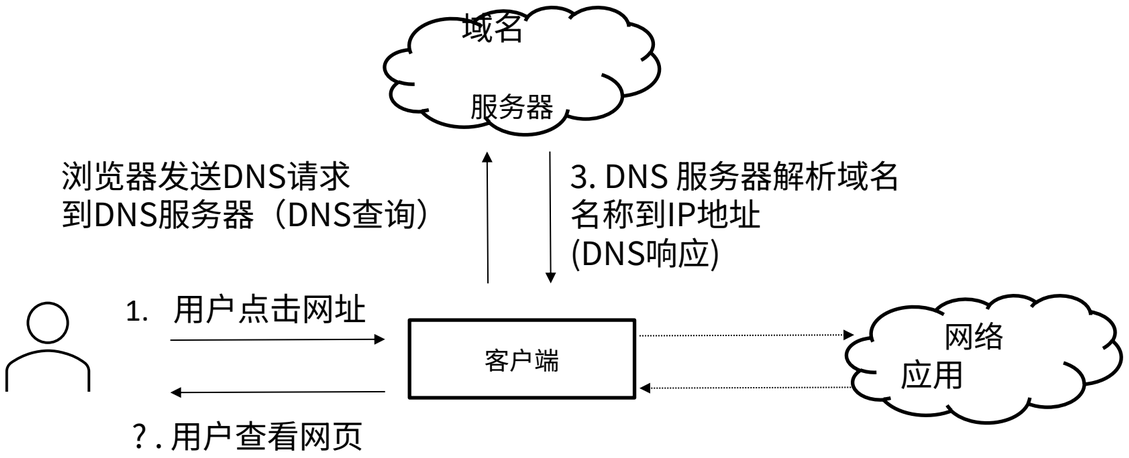
- Web应用程序的位置在哪里?
  - 域名 (例如：gla.ac.uk)，但互联网的语言是 IP 地址
  - DNS: 域名系统– DNS 服务器将翻译域名到IP地址

Clicking on a URL – DNS messages



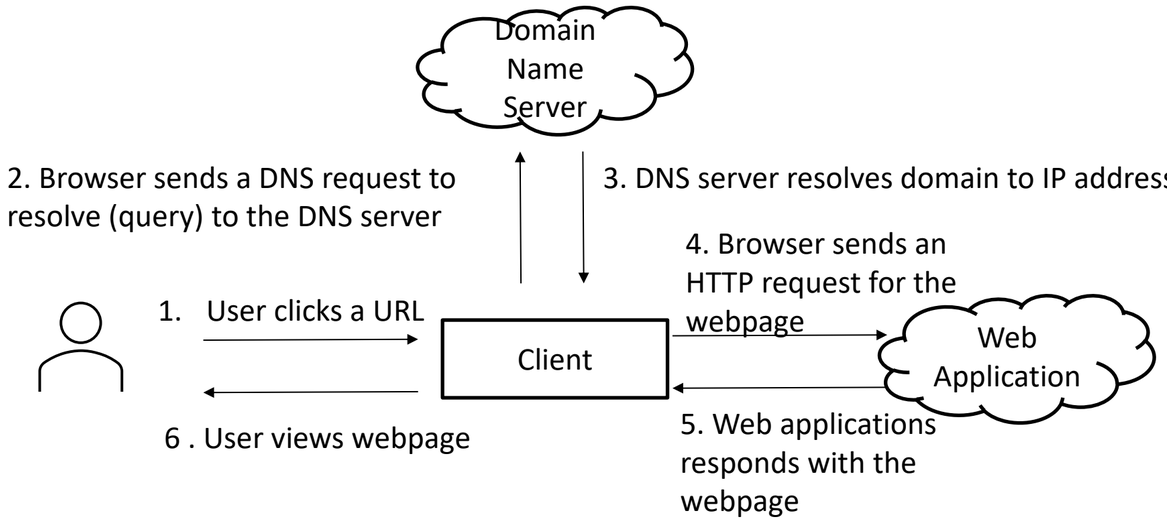
- The DNS server will respond with the IP address
  e.g. gla.ac.uk -> 130.209.16.90
- The **client** can now **send the HTTP request** to the web application

点击网址-DNS消息



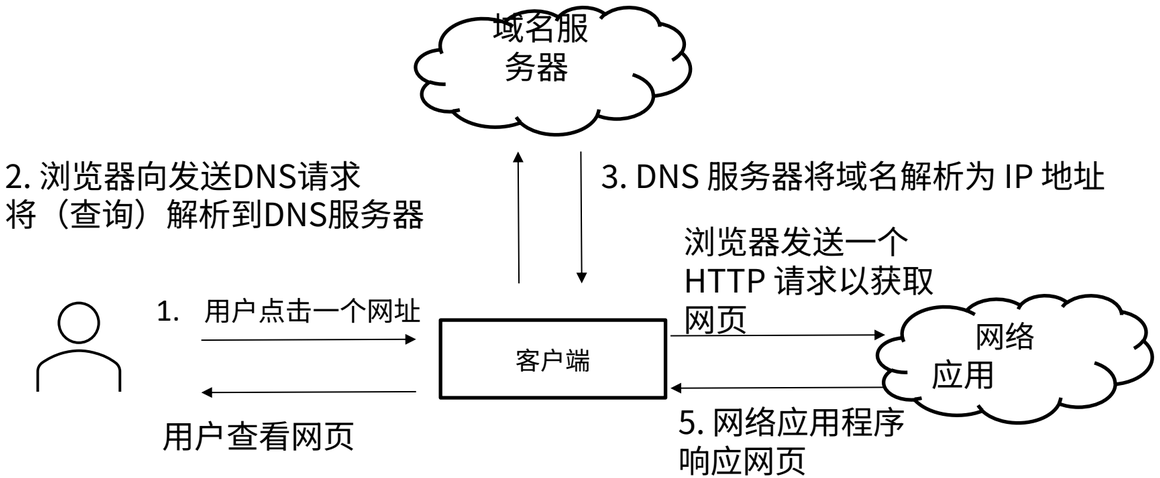
- DNS 服务器将响应 IP 地址
  例如：gla.ac.uk -> 130.209.16.90
- 客户端现在可以向web应用程序发送HTTP请求

Clicking on a URL – Flow of messages



- A few more things are happening under the hood...
- A connection needs to be established between the Client and the Web Application/Server -> TCP

点击网址–流量消息



- 还有一些事情正在后台进行...
- 需要在之间建立连接
客户端和Web应用程序/服务器 -> TCP

Requests and Responses (1)

The following happens when the **user agent** (i.e. the web browser/client) is asked to **send** a **message** (i.e. when the user clicks a link):

- First, the URL is turned into an **IP address**
  - **Request:** ask Domain Name System (DNS) for IP
  - **Response:** returns the IP for the URL
    - E.g. www.gla.ac.uk maps to 130.209.16.90

请求与响应 (1)

当 用户代理（即请求网页浏览器/客户端发送一条消息（即当用户点击链接时）：

- 首先，URL被转换为 IP地址
  - 请求： 向域名系统（DNS）请求IP
  - 响应： 返回该URL的IP
    - E.g. www.gla.ac.uk 映射到 130.209.16.90

Request and Responses (2)

- Second, a **TCP connection** is opened on a particular port on the node at that IP address
  - port 80 for HTTP (standard)
  - port 443 for HTTPS (encrypted through SSL)

- Next, a request is made using a specific URL Scheme (i.e. HTTP) and sent using that TCP connection.
  - **Request:** Get the home page of the specified URL.
  - **Response:** Returns the HTML for the home page

For a full ports list: https://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers

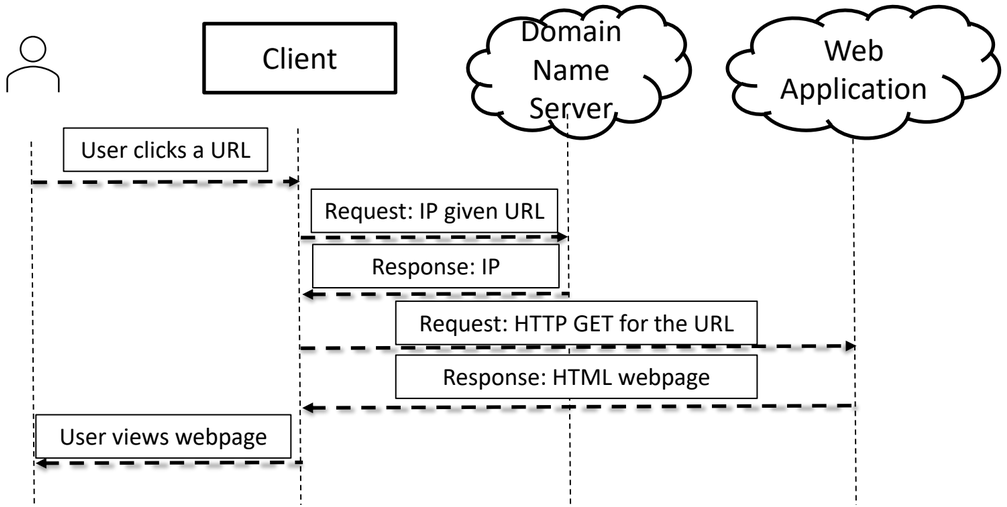
请求与响应 (2)

- 其次，在一个TCP 连接上打开该IP地址上节点的特定端口
  - 端口 80 用于 HTTP（标准）
  - 端口 443 用于 HTTPS（通过 SSL 加密）

- 接下来，使用特定的URL发出请求方案（即 HTTP）并通过该 TCP 发送连接。
  - 请求： 获取指定URL的主页。
  - 响应： 返回主页的HTML

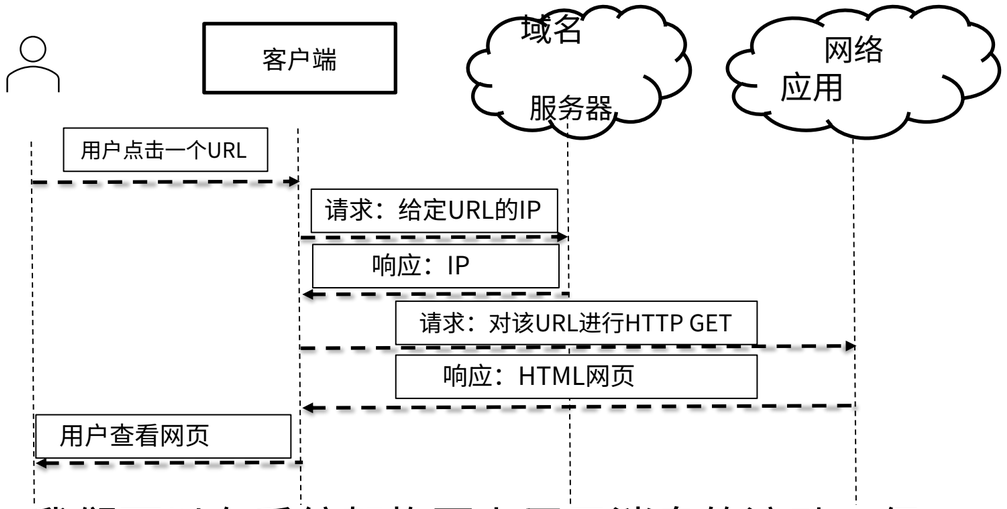
完整端口列表：https://en.wikipedia.org/wiki/List_of_TCP_and_UDP_port_numbers

Sequence Diagrams



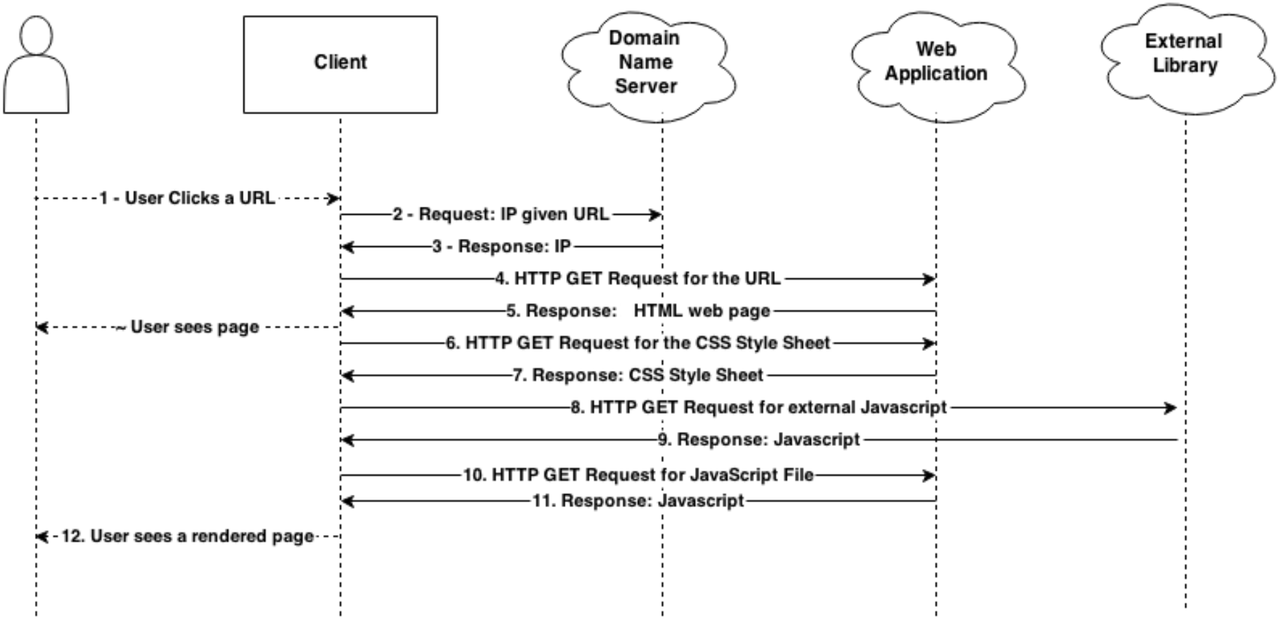
- We can show the flow of messages on the System Architecture Diagram, but many details are hidden
- Sequence Diagrams provide a better way to show the flow of messages
  - Label all messages

序列图



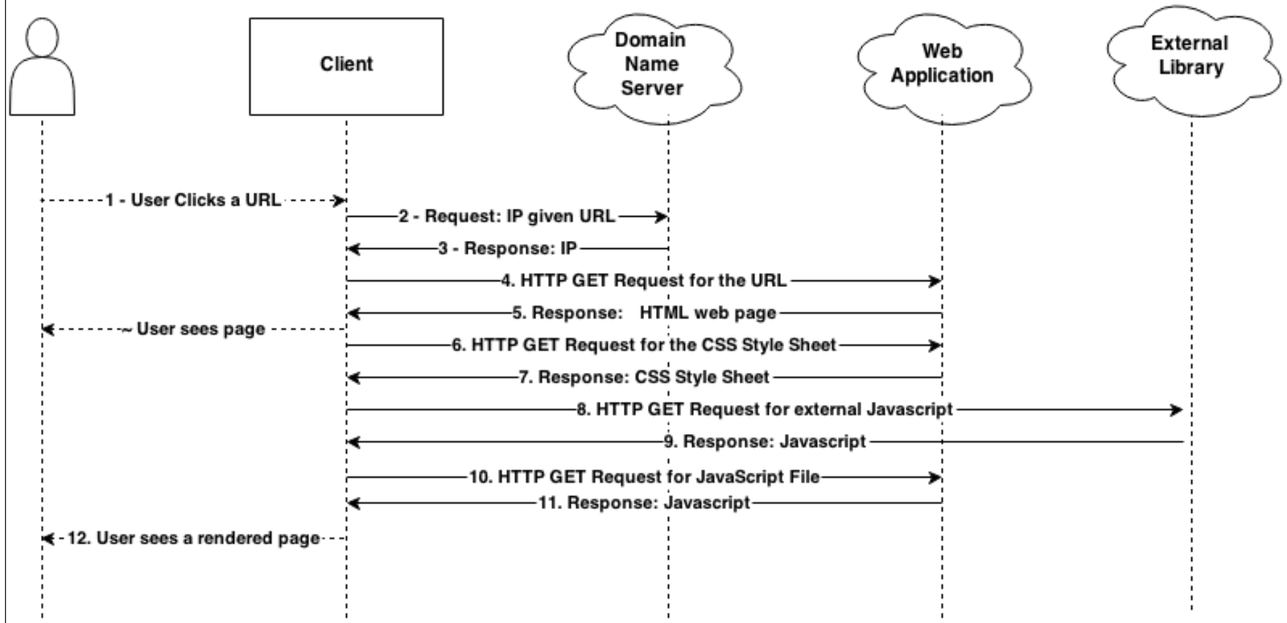
- 我们可以在系统架构图上展示消息的流动，但许多细节是隐藏的
- 序列图提供了一种更好的方式来展示流动消息的
  - 标记所有消息

Sequence Diagrams – an example



- Further requests for Style Sheets and Javascript

时序图– 一个示例



- 进一步请求样式表和JavaScript

Application-layer Protocols

- **Requests can be made using various protocols**
  - **http**: This is the most common protocol and indicates a file that a web browser can format and display - an HTML file, image file, sound file, etc.
  - **https**: Automatically performs Secure Socket Layer negotiation and always sends data in encrypted form
  - **file**: This indicates a file which is not in a recognised web format and will be displayed as text
  - Other protocols: ftp, mailto, news, telnet, ...

应用层协议

- **请求可以使用各种协议进行**
  - **http**: 这是最常见的协议，表示一个网页浏览器可以格式化和显示的文件 - 一个HTML文件、图像文件、声音文件等。
  - **https**: 自动执行安全套接字层协商，并始终以加密形式发送数据
  - 文件: 这表示一个不被识别的文件网页格式，将以文本形式显示
  - 其他协议：ftp、mailto、news、telnet、...

At the Application Layer

We shall be focusing on a protocol for communication and the payload formats:

1. **Hypertext Transfer Protocol (HTTP)**
   - The low level application protocol used to send messages
   - This is a specific URL Scheme
   - Follows a Request-Response pattern
   - Provides a number of ways to make a request (GET, POST)
2. **User Agent Specific Protocols (UASP)**
   - These package/wrap the information that will be sent when providing a response.
   - Such as XML, XHTML, JSON

在应用层

我们将专注于一种用于
通信的协议和有效载荷格式：

1. **超文本传输协议（HTTP）**
   - 用于发送消息的低级应用协议
   - 这是一个特定的URL方案
   - 遵循请求-响应模式
   - 提供多种请求方式（GET、POST）
2. **用户代理特定协议 (UASP)**
   - 这些包裹/包装了在提供响应时将要发送的信息。
   - 例如 XML、XHTML、JSON

HTTP

HTTP or **Hyper Text Transfer Protocol** is used to deliver virtually all files and other data using **8 bit characters**

– Usually, HTTP takes place through TCP/IP sockets

HTTP is used to transmit **resources**, not just files.

– A resource is some chunk of information that can be identified by a URL

- HTTP functions as a **request-response pattern** in the **client-server** computing model

HTTP

HTTP 或 超文本传输协议 用于
使用 8位

字符几乎传送所有文件和其他数据

– 通常，HTTP 通过 TCP/IP 套接字进行传输

HTTP用于传输 资源，而不仅仅是文件。

– 资源是可以被
通过URL识别

- **HTTP 作为一种 请求-响应模式 在**
客户端-服务器计算模型

HTTP Request and Response Messages

- Under HTTP, communication is as follows:
  - An HTTP client opens a connection and sends a **request message** to an HTTP server
    - Request messages are typically **GET** or **POST** methods
  - The server then returns a **response message**, usually containing the resource that was requested.
    - Respones are typically in XHTML, XML, but also in other formats like JSON
  - After delivering the response, the server **closes the connection** making HTTP a stateless protocol
    - i.e. not maintaining any connection information between transactions

HTTP请求和响应消息

- 在HTTP下，通信如下：
  - HTTP客户端打开连接并向HTTP服务器发送请求消息
    - 请求消息通常是GET或POST方法
  - 然后服务器返回响应消息，通常包含请求的资源。
    - 响应通常是XHTML、XML，但也可以是其他格式，如JSON
  - 在传递响应后，服务器关闭连接，使HTTP成为无状态协议
    - 即在事务之间不维护任何连接信息

HTTP GET (1)

- **HTTP GET**
  - GET retrieves data from the server
  - Should be used for operations that don't change server data

- How HTTP GET works
  - GET appends the data to the URL as key-value pairs
  - Syntax: URL ? key1 = value1 & key2 = value2
  - Example: https://www.google.com/search?q=itech
  - URL encoding: special characters within value are replaced
    - e.g. space is replaced by %20

HTTP GET (1)

- **HTTP GET**
  - GET 从服务器检索数据
  - 应在不更改服务器数据的操作中使用数据

- HTTP GET 的工作原理
  - GET 将数据作为键值对附加到 URL
  - 语法：URL ? key1 = value1 & key2 = value2
  - 示例：https://www.google.com/search?q=itech
  - URL 编码：值中的特殊字符被替换
    - 例如，空格被替换为 %20

HTTP GET (2)

- **User benefits:** The data is part of the URL – the user can see them, copy them, bookmark them, resubmit the page easily
- **When to use GET?** GET should be used for pages which **don't change anything on the server**
  - Example: Information requests
    - GET /user/profile?name=JaneDoe&address=123+Main+St HTTP/1.1
    - GET /search?query=lady+java HTTP/1.1

HTTP GET (2)

- 用户好处： 数据是URL的一部分– 用户可以看到它们，复制它们，轻松地收藏它们，重新提交页面
- 何时使用GET？ GET应在页面上使用 不会**更改服务器上的任何内容**
  - 示例：信息请求
    - GET /user/profile?name=JaneDoe&address=123+Main+St HTTP/1.1
    - GET /search?query=lady+java HTTP/1.1

HTTP POST (1)

- **HTTP POST**
  - Sends data to the server, packaged as part of the message
  - Should be used for programs that change data on the server
- How HTTP POST works
  - Unlike GET, POST includes data in the **message body**, not in the URL.
  - Required headers

    Content-Type: application/x-www-form-urlencoded

    Content-Length: 26 // number of characters
  - Message in the body

    name="xxx"&address="yyy"
  - The structure (paramName1 = "paramValue1"&paramName2=....) is defined by the *x-www-form-urlencoded* standard (Content-Type in the header)

HTTP POST (1)

- **HTTP POST**
  - 将数据发送到服务器，作为消息的一部分进行打包
  - 应当用于更改服务器上数据的程序
- HTTP POST 的工作原理
  - 与 GET 不同，POST 将数据包含在 消息体中，而不是在 URL 中。
  - 必需的头部

    Content-Type: application/x-www-form-urlencoded

    Content-Length: 26 // 字符数
  - 消息体中的内容

    name="xxx"&address="yyy"
  - 结构（paramName1 = "paramValue1"&paramName2=....）由*x-www-form-urlencoded* 标准（头部中的 Content-Type）定义

HTTP POST (2)

- **When to use POST?**
  - For operations with side effects
    - Database updates, sending emails etc
  - For file uploads and complex data
    - When multi-part/form data is required
  - For non-ASCII data
    - Handles special characters like accented letters more reliably
  - For large datasets
    - GET URLs can run into length limits (e.g. over 1KB)
  - To hide data from the user
    - Data is not visible in the URL, although the user can still view the page source

HTTP POST (2)

- **何时使用POST?**
  - 用于具有副作用的操作
    - 数据库更新、发送电子邮件等
  - 用于文件上传和复杂数据
    - 当需要多部分/表单数据时
  - 用于非ASCII数据
    - 更可靠地处理特殊字符，如重音字母
  - 用于大型数据集
    - GET URL可能会遇到长度限制（例如超过1KB）
  - 为了隐藏用户数据
    - 数据在URL中不可见，尽管用户仍然可以查看页面源代码

HTTP GET vs POST

- What is the difference between these methods?
- When would you use one over another?
  - A safe operation is an operation which does not change the data requested.
  - An idempotent operation is one in which the result will be the same no matter how many times you request it.

– Use GET for safe and idempotent requests
– Use POST for neither safe nor idempotent requests

HTTP GET与POST

- 这两种方法有什么区别?
- 什么时候会使用其中一种而不是另一种?
  - 安全操作是指不会改变请求数据的操作。
  - 幂等操作是指无论请求多少次，结果都是相同的操作。

– 对于安全且幂等的请求使用GET
– 对于既不安全也不幂等的请求使用POST

Differences between GET and POST

	GET	POST
Back button	Harmless	Data will be resubmitted
Bookmarked	Can be bookmarked	-
Cached	Can be cached	-
History	Parameters remain in history	-
Data visibility	Visible to everyone in the URL	Not visible
Data length/Data type	Max 2048 characters, only ASCII	No restrictions
Security	Less secure – data is part of the URL	More secure

GET与

POST之间的区别

	获取	POST之间的区别
返回按钮	无害	数据将被重新提交
已加书签	可以加书签	-
已缓存	可以缓存	-
历史	参数保留在历史记录	-
数据可见性	对URL中的所有人可见	不可见
数据长度/数据类型	最大2048个字符，仅限ASCII	没有限制
安全性	不太安全– 数据是URL的一部分	更安全

Other HTTP Methods

- **The HTTP protocol has numerous other methods too:**
  - **HEAD** is just like GET, except it asks the server to return the response headers only
    - useful to check characteristics of a resource without actually downloading it
  - **PUT** for storing data on the server
  - **DELETE** for deleting a resource on the server
  - **OPTIONS** for finding out what the server can do - e.g. switch to secure connections
  - **TRACE** for debugging connections
  - **CONNECT** for establishing a link through a proxy

其他HTTP方法

- **HTTP协议还有许多其他方法：**
  - **HEAD**与GET类似，只是它要求服务器仅返回响应头
    - 用于检查资源的特性，而不实际下载它
  - **PUT**用于在服务器上存储数据
  - **DELETE**用于删除服务器上的资源
  - **OPTIONS**用于查找服务器可以做什么 - 例如，切换到安全连接
  - **TRACE**用于调试连接
  - **CONNECT**用于通过代理建立链接

HTTP Response Status Codes

- HTTP responses return status codes in the 1st line of the server-to-client response message

- 200 OK
  – request succeeded, requested object later in this message
- 301 Moved Permanently
  – requested object moved, new location specified later in this message
- 400 Bad Request
  – request message not understood by server
- 404 Not Found
  – requested document not found on this server
- 505 HTTP Version Not Supported

ThePhoto by PhotoAuthor is licensed under CCYSA.

HTTP响应状态代码

- HTTP 响应在第一行返回状态码服务器到客户端响应消息的

- 200 OK
  – 请求成功，请求的对象在此消息后面
- 301 永久移动
  – 请求的对象已移动，新的位置将在此消息中稍后指定
- 400 错误请求
  – 请求消息未被服务器理解
- 404 未找到
  – 请求的文档在此服务器上未找到
- 505 HTTP版本不受支持

照片由PhotoAuthor授权，采用CCYSA许可。

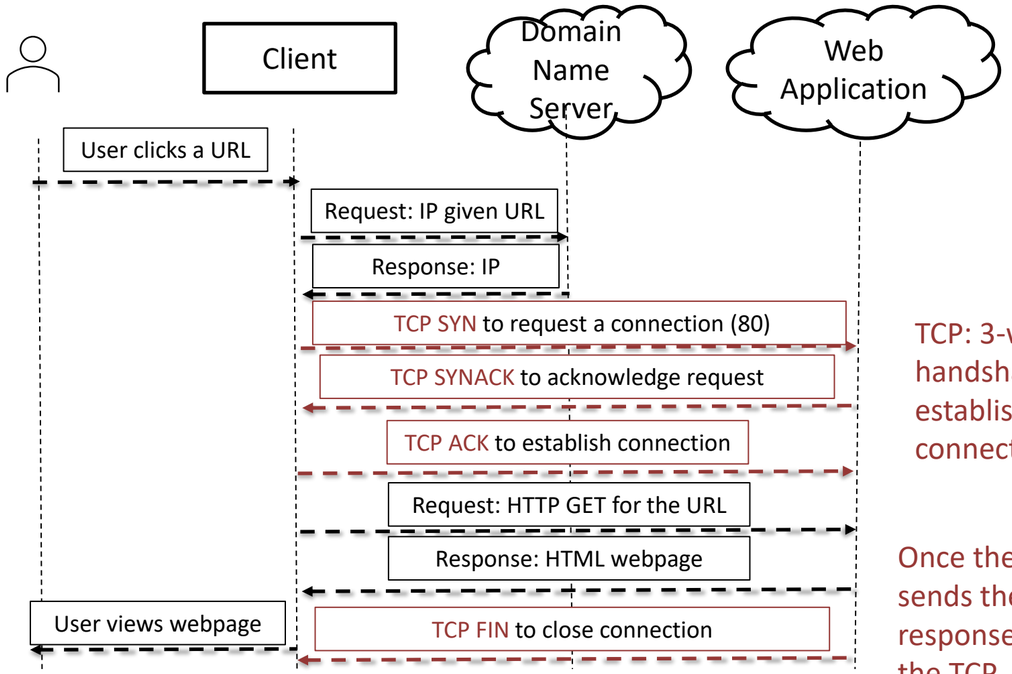
HTTP is stateless

- Under HTTP, communication is as follows:
  - An HTTP client opens a connection and sends a **request message** to an HTTP server
    - Typically the request is either a GET or POST
  - The server then returns a **response message**, usually containing the resource that was requested.
    - Typically, in XHTML, XML, but also in other formats like JSON
  - After delivering the response, the server **closes the connection** making HTTP a stateless protocol
    - i.e. not maintaining any connection information between transactions

HTTP是无状态的

- 在HTTP下，通信如下：
  - HTTP客户端打开连接并向HTTP服务器发送请求消息
    - 通常请求是GET或POST
  - 然后服务器返回响应消息，通常包含请求的资源。
    - 通常在XHTML、XML中，但也可以在其他格式如JSON中
  - 在传递响应后，服务器关闭连接，使HTTP成为无状态协议
    - 即不维护任何连接信息
      事务

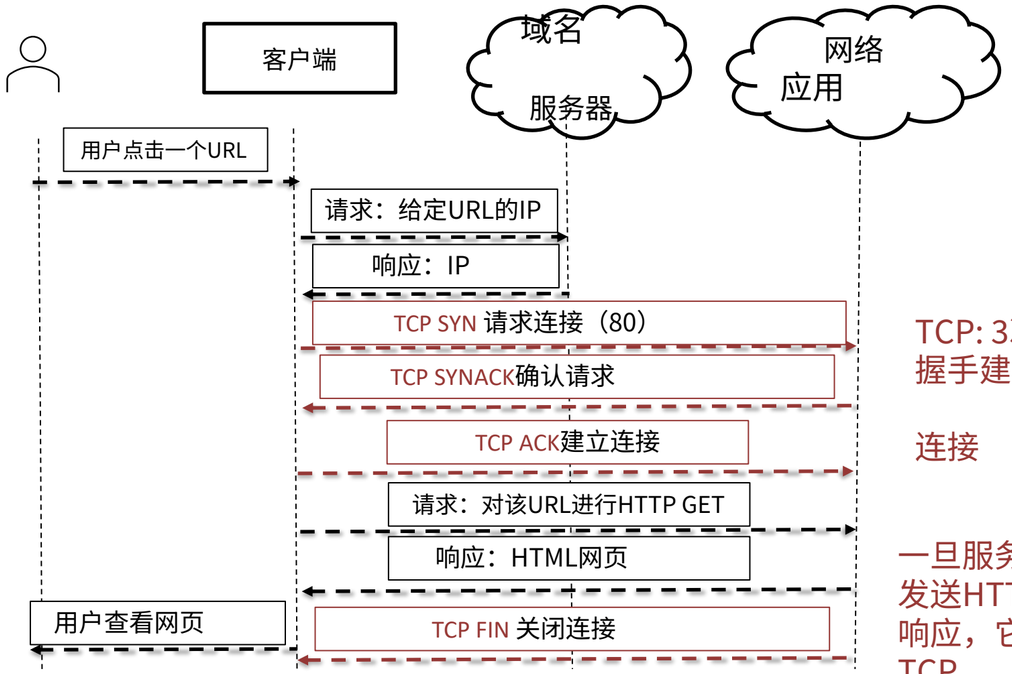
HTTP and TCP



TCP: 3-way handshake to establish connection

Once the server sends the HTTP response, it closes the TCP connection

HTTP和TCP



TCP: 3次握手建立连接

一旦服务器发送HTTP响应，它将关闭TCP连接

HTTP Statelessness can be a problem

- HTTP does not require the server to retain any information about the client/user
  - All requests are independent
- This is a problem if we want to **maintain a session**
  - Logged-in user
  - Multi-step forms
  - User preferences
  - E-commerce cart

HTTP无状态性可能是一个问题

- HTTP不要求服务器保留任何关于客户端/用户的信息
  - 所有请求都是独立的
- 如果我们想要维持一个会话，这就是一个问题
  - 已登录用户
  - 多步骤表单
  - 用户偏好
  - 电子商务购物车

Dealing with Statelessness

Common solutions to overcome statelessness

- Client-side
  - HTTP Cookies
    - See TwD Chapter 10
  - URL encoding to store a session-ID within a URL
- Server-side
  - Server-side Sessions
  - Hidden Form Fields

处理无国籍问题

克服无国籍问题的常见解决方案

- 客户端
  - HTTP Cookies
    - 参见 TwD 第10章
  - 在URL中编码以存储会话ID
- 服务器端
  - 服务器端会话
  - 隐藏表单字段

HTTP Cookies

- How do cookies work?
  - Alice uses browser on device to visit an e-commerce site for the first time
  - When the initial HTTP request arrives at the server, the server creates a **unique ID - the cookie** – and an entry in the backend database
  - Subsequent HTTP requests from Alice to this site will contain the cookie ID, allowing the server to identify Alice
- When to use?
  - Authorization, E-commerce shopping carts
- Privacy concerns
  - Websites can learn a lot about a user
  - Third-party cookies allow the same user to be **tracked across multiple websites**

When you open any website

https://i.redd.it/uh6nfugw5jf51.jpg

HTTP Cookies

- Cookies是如何工作的?
  - 爱丽丝第一次在设备上使用浏览器访问一个电子商务网站
  - 当初始HTTP请求到达服务器时，服务器会创建一个唯一ID - cookie - 并在后端数据库中生成一个条目
  - 爱丽丝后续向该网站发送的HTTP请求将包含cookie ID，从而允许服务器识别爱丽丝
- 何时使用?
  - 授权，电子商务购物车
- 隐私问题
  - 网站可以从用户身上学到很多信息
  - 第三方 cookies 允许相同的 **用户将在多个网站上被追踪**

When you open any website

https://i.redd.it/uh6nfugw5jf51.jpg

Persistent HTTP (1.1)

Persistent HTTP (HTTP1.1):

- The server leaves connection open after sending response
- Subsequent HTTP messages between same client/server sent over open connection
- Time to load a page: As little as one round-trip (request+response) time for all the referenced objects (cutting response time in half)

持久性HTTP (1.1)

持久性HTTP (HTTP1.1):

- 服务器在之后保持连接开放发送响应
- 后续的HTTP消息在相同的客户端/服务器之间通过开放连接发送
- 加载页面的时间：仅需一次往返所有引用对象的(request+response)时间（将响应时间缩短一半）

Further HTTP improvements

- What if the first request takes a really long time?
  - Fetching a large media file
  - All subsequent requests block -> low performance!
- HTTP 2 divides objects to be retrieved into segments
  - A response carrying a large media file is segmented
- HTTP 3 replaces TCP entirely with QUIC (Quick UDP Internet Connections) to improve RTT

进一步的HTTP改进

- 如果第一次请求耗时很长怎么办?
  - 获取大型媒体文件
  - 所有后续请求都会被阻塞 -> 性能低下!
- HTTP 2将要检索的对象分为多个段
  - 携带大型媒体文件的响应被分段
- HTTP 3完全用QUIC（快速UDP互联网连接）替代TCP，以改善RTT

Browser caching

- **Browser caching** stores some data on the browser to avoid frequent transfers from the server
- How does browser caching works?
  - Alice uses browser on device to visit an e-commerce site for the first time.
  - When the first HTTP request arrives at the server, the server responds with the requested resources (HTML, CSS, images, etc.) along with **caching headers**
  - Alice's browser stores these resources in its **local cache**
  - Subsequent HTTP requests from Alice include **conditional GET headers** to check whether the cached resource has changed.
  - If the resource hasn't changed, the server returns a **304 Not Modified** response, allowing the browser to load the resource from the cache without downloading it again.

- When to use?
  - To **improve page load speed**, and reduce server load
- Concerns
  - **Staleness!** If not used properly, caching can result in serving outdated content

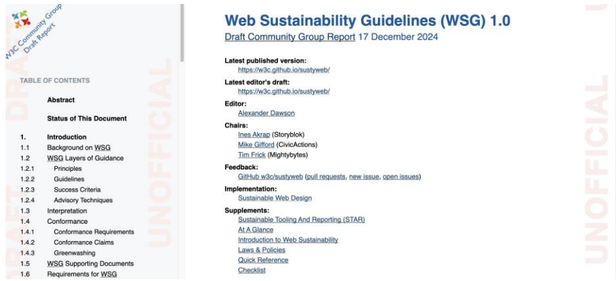
浏览器缓存

- 浏览器缓存在浏览器上存储一些数据，以避免频繁从服务器传输
- 浏览器缓存是如何工作的?
  - Alice第一次在设备上使用浏览器访问电子商务网站。
  - 当第一个HTTP请求到达服务器时，服务器会做出响应请求的资源（HTML、CSS、图像等）以及缓存头
  - Alice的浏览器将这些资源存储在其本地缓存中
  - 后续来自爱丽丝的HTTP请求包含条件GET头以检查缓存的资源是否已更改。
  - 如果资源没有变化，服务器将返回**304 Not Modified** 响应，允许浏览器从缓存中加载资源，而无需再次下载。

- 何时使用?
  - 为了 提高页面加载速度，并减少服务器负载
- 关注
  - 过时！ 如果使用不当，缓存可能导致提供过时的内容

Web Sustainability Guidelines

- Web Sustainability Guidelines: a working draft from the W3C



https://w3c.github.io/sustyweb/

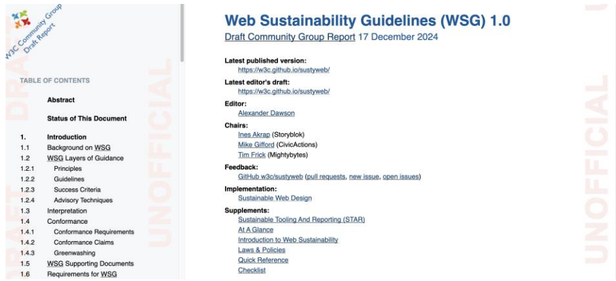
12 guidelines relevant to Hosting and Infrastructure (Section 4):
https://sustainablewebdesign.org/guideline-categories/hosting-infrastructure-and-systems/

Every HTTP request will result to a response (data transfer) from the server to the client.

Data transfers over the network consume a lot of energy!

网络可持续性指南

- 网络可持续性指南：来自W3C的工作草案



https://w3c.github.io/sustyweb/

与托管和
基础设施（第4节）相关的12条指南：
https://sustainablewebdesign.org/guideline-categories/hosting-基础设施和系统/

每个HTTP请求都会导致从服务器到客户端的响应（数据传输）。

数据传输通过网络消耗很多能量！

Web sustainability and Messaging

- **Guideline #4.2**: Optimize Browser Caching
  - Browser caching reduces the requirement for files to need to be constantly reloaded from the server
  - Significant reduction in data transfers over the network (between the client side and the server side)
  - High impact in energy consumption and making the web greener

网络可持续性和消息传递

- 指南 #4.2: 优化浏览器缓存
  - 浏览器缓存减少了文件需要不断从服务器重新加载的要求
  - 显著减少网络上的数据传输(在客户端和服务器端之间)
  - 对能源消耗产生重大影响，使网络更加环保

Web sustainability and Messaging

- **Guideline #4.3**: Compress your files
  - Remember that, with persistent HTTP, the time to load a page is relevant to the objects being loaded
  - Files/Data/Media need to be transferred to every visitor, incurring significant load on the network
  - Aligns well with the mobile-first principle (see Week 3) - compress media files and offer different resolutions for different devices

网络可持续性和消息传递

- 指南 #4.3: 压缩您的文件
  - 请记住，在持久HTTP中，加载页面的时间与正在加载的对象相关
  - 文件/数据/媒体需要传输给每个访客，这会对网络造成显著负担
  - 与移动优先原则（见第三周）非常契合 - 压缩媒体文件并为

    不同设备提供不同分辨率

Web sustainability and Messaging

- **Guideline #4.4**: Use Error Pages and Redirects Carefully
  - Ensure links are correct so that HTTP requests won't fail
  - Redirect pages only when necessary

网络可持续性和消息传递

- 指导原则 #4.4: 小心使用错误页面和重定向
  - 确保链接正确，以免HTTP请求失败
  - 仅在必要时重定向页面

Web sustainability and Messaging

- **Guideline #4.7**: Maintain a relevant refresh frequency
  - Only send data from the server when the visitor needs it
  - Rely on client-side caching, or server-side caching as much as possible
  - Prefer manual refreshes

网络可持续性和消息传递

- 指导原则 #4.7: 维持相关的刷新频率
  - 仅在访客需要时从服务器发送数据
  - 尽可能依赖客户端缓存或服务器端缓存尽可能
  - 偏好手动刷新

Inter-Process Protocols

进程间协议

Protocols for Inter-Process Communication (IPC)

- HTTP and the other URL schemes are based on the user-agent/client-server model of data
  - Look at Django: it may be a middleware, but it still features a web server (built-in development web server or Nginx/Apache in deployment)
- What about applications in which the code has been distributed about the internet in other ways?
  - e.g. web services, The Cloud
- Solution: Inter Process Communication Protocols
  - IPC techniques are divided into methods for:
    - **message passing**,
    - **synchronization**,
    - **shared memory**, and
    - **remote procedure calls** (RPC)

Protocols for 进程间通信 (IPC)

- HTTP及其他URL方案是基于用户-代理/客户端-服务器模型的数据
  - 看看Django：它可能是一个中间件，但它仍然具有一个网络服务器（内置开发网络服务器或Nginx/Apache 在部署中)
- 那么，代码以其他方式在互联网上分发的应用程序呢?
  - 例如：网络服务，云计算
- 解决方案：进程间通信协议
  - IPC技术分为以下几种方法：
    - **消息传递**,
    - **同步**,
    - **共享内存，以及**
    - **远程过程调用**(RPC)

Legacy methods for IPC

Some legacy techniques:

- **SOAP** – the standard method for Web Services
  - Still used in many legacy systems, necessary to maintain
  - Quite complex to implement
- **XML-RPC** – an XML protocol for describing method calls
  - Simpler and more lightweight than SOAP
  - Can still be found in some older systems

传统的IPC方法

一些传统技术：

- **SOAP**– 网络服务的标准方法
  - 仍在许多遗留系统中使用，必须维护
  - 实现起来相当复杂
- **XML-RPC** – 一种用于描述的XML协议方法调用
  - 比SOAP更简单、更轻量
  - 仍然可以在一些旧系统中找到

Modern IPC

Modern techniques:

- **RPC:** Remote procedural calls
  - A method running on one machine can call a method running on another
  - gRPC is the most popular framework (created by Google, now open-source) on top of HTTP/2
  - Used at the core of many distributed applications
- **REST:** Representational State Transfer
  - Uses simple HTTP methods to exchange XML or JSON
  - RESTful APIs are implementations of REST – you can build one on your own!
    - Django REST Framework
  - Stateless, easy to integrate, scalable, include features like authentication and caching

现代IPC

现代技术：

- 远程过程调用Remote procedural calls
  - 一个在一台机器上运行的方法可以调用在另一台机器上运行的方法
  - gRPC是最流行的框架（由谷歌创建，现在基于HTTP/2的开源
  - 用于许多分布式应用的核心
- **REST:** 表现层状态转移
  - 使用简单的HTTP方法交换XML或JSON
  - RESTful APIs 是 REST 的实现– 你可以在上面构建一个你自己的!
    - Django REST Framework
  - 无状态、易于集成、可扩展，包含身份验证和缓存等功能

gRPC

- gRPC is a modern, open-source RPC framework designed for high-performance communication
- gRPC is built on top of HTTP/2
  - Benefits from multiplexing transmission (load many resources in parallel), compressing headers, and the better connection management
- gRPC supports communication between micro-services written in different languages
  - There is a description of the API contracts (Protocol Buffer)
  - Server and client use stub codes for the methods (Protcol Buffer compiler and gRPC plugins)

gRPC

- gRPC 是一个现代的开源 RPC 框架设计用于高性能通信
- gRPC建立在HTTP/2之上
  - 从多路复用传输中获益（加载多个并行资源)、压缩头部，以及更好的连接管理
- gRPC支持微服务之间的通信用不同语言书写
  - 有关于API合约（协议缓冲区）的描述
  - 服务器和客户端使用存根代码来实现这些方法（协议缓冲区编译器和 gRPC 插件)

gRPC Example

- Client side:

The client uses generated code to call `GetStockQuote("IBM")`

- Server side:

The server implements the `GetStockQuote` method, processes the request and returns the stock price

- Data structures are defined in the `.proto` file, and the Protocol Buffer compiler will generate source code for languages like C++, Python, Go, Java and more

```
syntax = "proto3";

package stock;

// Service definition for obtaining a stock quote.
service StockService {
  // RPC method to get a stock quote for a given symbol.
  rpc GetStockQuote (StockRequest) returns (StockResponse) {}
}

// Request message containing the stock symbol.
message StockRequest {
  string symbol = 1;
}

// Response message containing the stock price.
message StockResponse {
  double price = 1;
}
```

gRPC 示例

- 客户端：

客户端使用生成的代码调用`GetStockQuote("IBM")`

- 服务器端：

服务器实现了 `GetStockQuote` 方法，处理请求并返回股票价格

- 数据结构在 `.proto` 文件中定义，协议缓冲区编译器将为 C++、Python、Go、Java 等语言生成源代码

```
syntax = "proto3";

package stock;

// 获取股票报价的服务定义。
服务 StockService {
  // RPC 方法获取给定的股票报价符号。
  rpc GetStockQuote (StockRequest) returns (StockResponse) {}
}

// 请求包含股票代码的消息。
message StockRequest {
  string symbol = 1;
}

// 响应消息包含股票价格。
消息 StockResponse {
  double price = 1;
}
```

REpresentational State Transfer

- REST is a rather different way of using Web Services
- It avoids specific protocols, rather expecting:
  - the service to send a file/resource, and,
  - the client to pick it apart (using SAX, DOM, XSLT, etc.) – *more in Lecture 6/Client-side scripting*
- REST builds directly on HTTP
  - sending simple GET or POST requests and expecting the result to be XML/JSON/ETC rather than XHTML

表示性状态转移

- REST是一种使用Web服务的相当不同的方式
- 它避免了特定的协议，而是期望：
  - 服务发送一个文件/资源，并且，
  - 客户端将其解析（使用SAX、DOM、XSLT，等）– *更多内容见第6讲/客户端脚本*
- REST直接建立在HTTP之上
  - 发送简单的GET或POST请求，并且期望结果为XML/JSON/等
    比 XHTML 更

Why REST

- It is simpler than SOAP or XML-RPC,
- In comparison to SOAP and XML-RPC, it is designed to transport resources
- Requires some additional coding to handle either end
  - i.e. XML Parsing
- In a sense, HTTP itself is a **ReSTful** application, with the HTTP being interpreted by the server and user agent

为什么选择REST

- 它比SOAP或XML-RPC更简单，
- 与SOAP和XML-RPC相比，它旨在传输资源
- 需要一些额外的编码来处理任一端
  - 即XML解析
- 从某种意义上说，HTTP本身就是一个**ReSTful** 应用程序，HTTP由服务器和用户代理进行解释

REST Example

- Assume that the resource is represented by http://www.example.org/stock/IBM
- The server should return data in XML format

```
import xml.etree.ElementTree as ET
from urllib.request import urlopen

# Define the RESTful API endpoint for the stock quote
url = "http://www.example.org/stock/IBM"

# urlopen will generate a GET request
with urlopen(url) as response:
    xml_data = response.read()

# Parse the XML data
root = ET.fromstring(xml_data)
# Assuming the XML structure contains <price> within <stock>
price = root.find('price').text
```

Request Message

Response Message

REST 示例

- 假设资源由 http://www.example.org/stock/IBM 表示
- 服务器应返回 XML 格式的数据

```
import xml.etree.ElementTree as ET
from urllib.request import urlopen

# 定义股票报价的RESTful API端点
url = "http://www.example.org/stock/IBM"

# urlopen 将生成一个 GET 请求
with urlopen(url) as response:
    xml_data = response.read()

# 解析 XML 数据
root = ET.fromstring(xml_data)
# 假设 XML 结构中包含 <price> 在 <stock> 内
price = root.find('price').text
```

请求消息

响应消息

ReST Examples in Python

ReST 示例 在 Python 中

Search Spotify using GET

```
from urllib.request import urlopen
from urllib.error import URLError

url = ‘https://api.spotify.com/v1/search?type=artist&q=snoop’

try:
   response = urlopen( request )
   print(response.read( 1000 ))

except URLError as err:
   print(err.reason)
```

No params required, generates a GET request

WARNING: A proxy will need to be used if you try this in the university and you need an OAUTH2 key to access it

使用GET搜索Spotify

```
from urllib.request import urlopen
from urllib.error import URLError

url = ‘https://api.spotify.com/v1/search?type=artist&q=snoop’

try:
   response = urlopen( request )
   print(response.read( 1000 ))

except URLError as err:
   print(err.reason)
```

不需要参数，生成一个GET请求

警告：如果您在大学中尝试此操作，则需要使用代理，并且您需要一个OAUTH2密钥才能访问它

Add to a track to a playlist using POST

```python
import json
from urllib.request import Request, urlopen

# Define the API endpoint for creating a playlist
url =“http://api.musicfun.com/v1/playlists”

playlist_data = {
    "name": "My Awesome Playlist",
    "description": "A collection of my favorite tunes.",
    "tracks": ["track1", "track2", "track3"]
}

# Convert the playlist data to a JSON string and encode it to bytes
encoded_data = json.dumps(playlist_data).encode('utf-8')

# Create a Request object with the JSON data (this makes it a POST request)
req = Request(url, data=encoded_data)
req.add_header("Content-Type", "application/json")

# Send the POST request and read the response
with urlopen(req) as response:
    result = response.read().decode('utf-8')

print("Response from MusicFun API:")
print(result)
```

The Request object now has some encoded data – this is a POST request

使用

POST将曲目添加到播放列表

```python
import json
from urllib.request import Request, urlopen

# 定义创建播放列表的API端点
url =“http://api.musicfun.com/v1/playlists”

playlist_data = {
    "name": "我的精彩播放列表",
    "description": "我最喜欢的曲目的集合.",
    "tracks": ["track1", "track2", "track3"]
}

# 将播放列表数据转换为JSON字符串并编码为字节
encoded_data = json.dumps(playlist_data).encode('utf-8')

# 创建一个包含JSON数据的Request对象（这使其成为POST请求）
req = Request(url, data=encoded_data)
req.add_header("Content-Type", "application/json")

# 发送POST请求并读取响应
with urlopen(req) as response:
    result = response.read().decode('utf-8')

print("来自MusicFun API的响应:")
print(result)
```

Request对象现在有一些编码的数据– 这是一个POST请求

Summary

- Networking basics
  - OSI vs TCP/IP models – layered architecture for web communication
  - Client-Server model – clients request resources from servers (web servers, middleware)
- HTTP fundamentals
  - Request/Response pattern
  - GET to retrieve data, POST to send data
  - Stateless protocol: HTTP transactions are independent
- Inter-Process Communication
  - REST: Lightweight, resource-oriented, built on standard HTTP methods
- Extras:
  - Performance/Sustainability: persistent connections, modern HTTP, caching
  - Real-world applications: RESTful APIs and gRPC

摘要

- 网络基础
  - OSI与TCP/IP模型– 网络通信的分层架构
  - 客户端-服务器模型– 客户端向服务器请求资源（网络服务器、中间件）
- HTTP基础
  - 请求/响应模式
  - 使用GET获取数据，使用POST发送数据
  - 无状态协议：HTTP事务是独立的
- 进程间通信
  - REST：轻量级、面向资源，基于标准HTTP方法构建
- 额外信息：
  - 性能/可持续性：持久连接，现代HTTP，缓存
  - 实际应用：RESTful API 和 gRPC

Half-way there…

- Next week (Week 5), you will be about half-way through the course
  - One quiz done, two to go
  - First part of group coursework done, second part to go
  - Keep up the good work!
- Use communication channels! Come to the labs! Ask questions!
  - Padlet is your friend!
- See you on Week 6!

半途而已…

- 下周（第5周），你将大约完成一半通过课程
  - 一场测验完成，还有两场待进行
  - 小组课程的第一部分完成，第二部分待完成去
  - 继续保持良好的工作！
- 使用沟通渠道！来实验室！提问吧！
  - Padlet是你的朋友！
- 第六周见！